The MLP labs: a first task, then an MLP on CIFAR-10, E0–E15

AI Methods for Science (77222) · Part I, weeks 1–2

Part A: first task, fit a function (20 minutes)

Write the whole thing yourself: model, loss, optimizer, training loop. No given code.

Data. $n = 200$ points, $x \sim U[0, 1]$, $y = \sin(2\pi x) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, $\sigma = 0.25$. A test set: 200 fresh points from the same distribution.

Model and training. MLP 1–64–64–1 with `tanh`; `nn.MSELoss`; `torch.optim.Adam`, `lr=1e-2`; full batch; 2000 steps.

1. Write the model (`nn.Sequential`), the loss, the optimizer and the five-line loop. Print the loss every 200 steps.
2. Plot the data, the true function and the fit.
3. Compute the train and the test MSE. Compare both with the noise floor $\sigma^2 = 0.0625$: which of the two can go below it, and what does it mean when it does?

Then try (one line each in your report):

- plain SGD and SGD with `momentum=0.9` instead of Adam, same number of steps;
- hidden width 1 instead of 64;
- 5000 steps instead of 2000;
- $n = 15$ training points and 5000 steps.

Stretch. Two interleaved spirals in 2D, two classes: MLP 2–64–64–2 with ReLU, `nn.CrossEntropyLoss` on the logits, Adam. Plot the decision regions. Same loop; only the output dimension and the loss change.

Things that go wrong. `x` of shape `(200,)` instead of `(200, 1)`; a target of shape `(200,)` against an output `(200, 1)` (`MSELoss` broadcasts to 200×200 and only warns); a missing `zero_grad()`; storing `loss` instead of `loss.item()`.

Part B: an MLP on CIFAR-10

The setup

You get a working network and a working training procedure: `mlp_lab.py`. One function,

```
result = train(config)
```

takes a dictionary of hyperparameters and returns a dictionary of measurements:

- `result["history"]`: per epoch, `train_loss`, `train_acc`, `val_loss`, `val_acc`, `seconds`;
- `result["summary"]`: `n_params`, `best_epoch` (lowest validation loss), `val_acc_at_best`, `max_val_acc`, final train/validation loss and accuracy, `sec_per_epoch`, `diverged`, and `test_acc` if requested.

The hyperparameters, with defaults (`python mlp_lab.py -show-defaults` prints them):

key	default	meaning
`hidden`	`[512, 256]`	widths of the hidden layers; `[]` is a linear classifier
`activation`	`"relu"`	`"relu"`, `"sigmoid"` or `"tanh"`
`lr`	`0.05`	learning rate of plain SGD
`weight_decay`	`0.0`	L_2 penalty $\frac{\lambda}{2}\|\theta\|^2$ added to the loss (SGD's `weight_decay`)
`dropout`	`0.0`	dropout probability after each hidden activation; off at evaluation
`batch_size`	`128`	minibatch size
`epochs`	`20`	passes over the training set
`train_fraction`	`1.0`	use only this fraction of the 45,000 training images
`normalize`	`true`	standardize the pixels, or feed raw values 0..255
`seed`	`0`	initialization and shuffling (the train/validation split never changes)
`test`	`false`	also report test accuracy. **Once, at the end (E11)**
`device`	`"cpu"`	`"cpu"`, `"cuda"`, `"mps"` or `"auto"`

Your job is the *experiments*: decide what to run, run it, collect the numbers, plot them, and explain what you see.

Rules

- **Do not edit `mlp_lab.py`.** Everyone runs the same network, so tables are comparable across the room.
- **Use an AI assistant for the pipeline** (E0). Writing loops, file handling and plotting code is not the point of the lab.
- **The numbers have to be yours**, measured on your machine. **The explanations have to be yours**: before each experiment, write one line saying what you expect; afterwards, say whether you were right and why.

Set up

1. Get the lab folder from the lab repository, https://github.com/yohai/aims-lab-session2 (clone, or Code → Download ZIP).
2. You need Python ≥ 3.11 with the packages in `requirements.txt` (PyTorch, torchvision, numpy, pandas, matplotlib). Install them with whatever you use: `pip install -r requirements.txt` in a venv or conda environment, or `uv run mlp_lab.py ...`, which installs them by itself. Ask your ai chatbot if unsure.
3. Inside the folder, run
   `python mlp_lab.py -download` (downloads CIFAR-10, 170 MB; can take half an hour on a slow connection)
   `python -c "import mlp_lab; mlp_lab.train({'epochs': 1})"` (one epoch, to check)

Do the download at home, not in class, and bring a laptop with it working. Times below are for a laptop CPU at ≈ 1.5 s per epoch of the default network. Yours may be 2–3× slower. If so, cut the epochs and say so in your table.

Reporting your results

Summarize your results in one short report per group, as a PDF (from a document, Markdown, LaTeX or a notebook export). For each experiment: your one-line prediction, the table, the figure(s), and two or three sentences of explanation.

Submit it as a GitHub issue on the lab repository, one issue per group: open
https://github.com/yohai/aims-lab-session2/issues/new/choose
choose *Lab report*, attach the PDF, fill in the group members and your best config with its test accuracy.

Note that GitHub issues are public and will be visible to anyone on the internet. I will delete them when the course is over.

The experiments

E0. Build the pipeline

with your AI assistant, ∼15 min

Ask your assistant for a small tool that

1. takes a named experiment, i.e. a list of configs (e.g. a base config plus one hyperparameter swept over values);
2. runs `mlp_lab.train` on each config and saves every result to disk (one JSON per run);
3. skips runs that are already saved, so an interrupted sweep can be resumed (key the saved file on the whole config, not on a name you type: a key that omits a hyperparameter returns an old run for a new config);
4. keeps a run whose loss became `nan` (its history then holds `None`): record it and show it, do not crash or drop it;
5. collects all runs into one table: config columns + summary columns (a CSV or a pandas DataFrame);
6. plots (a) any `history` curve for several runs on one axis, and (b) any summary quantity against any hyperparameter.

Answer. Check the pipeline against one run you did by hand: same seed, same numbers?

E1. How big is the noise?

∼1.5 min

Run. the default config, `seed` $\in \{0, 1, 2\}$.

Collect. `val_acc_at_best` and `max_val_acc` for each seed.

Answer. How much do they spread? From now on, which difference between two configs would you believe?

E2. The learning rate

∼2 min

Run. `lr` $\in \{0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.0\}$, 10 epochs.

Collect. training-loss curves of all runs on one axis (log y); validation accuracy vs. epoch, same runs; `max_val_acc` vs. `lr` (log x).

Answer. Label each run: crawls, works, noisy, diverges. The training loss is an average over the epoch and looks smooth; the noise shows in the validation curve. What is the largest learning rate that does not blow up? Where is the best one relative to it?

E3. Width and depth

∼3 min

Run. `hidden` $\in \{$`[]`, `[64]`, `[512]`, `[2048]`, `[512,256]`, `[1024,1024,1024]`, `[256]*6`$\}$, 15 epochs.

Collect. `n_params`, `sec_per_epoch`, `val_acc_at_best`, `final_train_acc`; plot the last two against `n_params` (log x).

Answer. What does a linear classifier reach? Per parameter, does width or depth buy more? Is any difference you see larger than the seed spread of E1? If not, say so (or run three seeds for the two networks you want to compare). Does training accuracy predict validation accuracy?

E4. Train for too long

∼1.5 min

Run. the default config, 60 epochs.

Collect. train and validation loss vs. epoch; train and validation accuracy vs. epoch.

Answer. At which epoch is the validation loss lowest? What happens to it afterwards, and to the validation *accuracy*? Why can loss and accuracy disagree? Which of the two would you stop on?

E5. Batch size ~2 min

Run. `batch_size` $\in \{16, 128, 1024\}$ at `lr` $= 0.05$; then `batch_size` $= 1024$ at `lr` $= 0.4$. 10 epochs.

Collect. validation accuracy against three different x-axes: epochs, gradient steps (`epoch` $\times$ `steps_per_epoch`), and wall-clock seconds.

Answer. Which batch size wins? The answer depends on the x-axis: explain why. What did raising the learning rate for the large batch do?

E6. How much data ~5 min

Run. `train_fraction` $\in \{0.01, 0.03, 0.1, 0.3, 1.0\}$ with `epochs` $= \min(400, \mathrm{round}(20/\mathrm{fraction}))$, so that the runs take about the same number of gradient steps (the two smallest fractions hit the 400-epoch cap and get fewer steps; note it when you compare).

Collect. `max_val_acc` and `final_train_acc` vs. the number of training images (log x).

Answer. How does the gap between training and validation accuracy depend on the amount of data? Extrapolate: what would 10$\times$ more data give?

E7. Activation and depth ~2 min

Run. `activation` $\in$ {relu, sigmoid, tanh} $\times$ `hidden` $\in$ {`[256,256]`, `[256]*8`}, 15 epochs.

Collect. training loss vs. epoch, all six runs on one axis.

Answer. Which combinations train slowly, or not at all? Explain it with the slide on gradients through depth (the backward pass is a product of one factor per layer).

E8. No normalization ~1 min

Run. `normalize` $=$ `false`, `lr` $\in \{0.05, 0.001, 0.00001\}$, 10 epochs; compare with the default.

Collect. training loss vs. epoch.

Answer. Why does raw-pixel input fail at the default learning rate? (The first layer's gradient is $\boldsymbol{\delta}^{(1)}\mathbf{x}^\top$.) The loss need not become `nan`: an explosion in epoch 1 followed by a flat $\ln 10 \approx 2.30$, chance level, is the same failure. Can a smaller learning rate alone fix it?

E9. Weight decay ~5 min

Run. `weight_decay` $\in \{0, 10^{-4}, 10^{-3}, 10^{-2}\}$, 60 epochs (the 0 run is E4).

Collect. validation loss vs. epoch and validation accuracy vs. epoch, all runs on one axis each; `max_val_acc` and `final_train_acc` vs. `weight_decay` (log x; put 0 at the left).

Answer. Does the validation loss still turn upward, and when? Which λ underfits (training accuracy well below 100%)? Did the best validation accuracy move by more than E1's spread?

E10. Dropout ~3 min

Run. `dropout` $\in \{0, 0.2, 0.5\}$, 60 epochs (the 0 run is E4).

Collect. training and validation accuracy vs. epoch, all runs on one axis.

Answer. What happens to the training accuracy, and to the gap between the two curves? Why is a dropout run's training curve noisier? (A different network every step; the training accuracy is measured with dropout on.) Did the validation accuracy improve by more than E1's spread?

E11. Your best MLP

open

Using only the hyperparameters above, find the config with the highest `val_acc_at_best`. Choose it by validation accuracy only. Then run that one config **once** with `test = true` and report the test accuracy. The workshop target (sessions 5–6) is 54% on the test set; with loss-based early stopping the validation number may stay just below it while `max_val_acc` is above. Report both, and say which epoch the test accuracy belongs to.

Answer. Why is the test accuracy run once and not used to choose? Is your test accuracy above or below your validation accuracy, and is the difference within the noise of E1?

Part C: the same MLP, with the week-2 knobs

The setup

`mlp_lab2.py` is `mlp_lab.py` with new hyperparameters. Same function, same results format, same defaults: `train({})` gives exactly the numbers of E1. Your pipeline from E0 works unchanged after `from mlp_lab2 import train`. The new keys (`python mlp_lab2.py -show-defaults` prints all of them):

key	default	meaning
`optimizer`	`"sgd"`	`"sgd"`, `"adam"` or `"adamw"`
`lr`	`0.05`	the (peak) learning rate. Plain SGD wants ~ 0.05 here, momentum 0.9 ~ 0.005, Adam ~ 0.001
`momentum`	`0.0`	SGD only: $v \leftarrow \beta v + g$, $\theta \leftarrow \theta - \eta v$
`weight_decay`	`0.0`	$\lambda\theta$ added to the gradient (`sgd`, `adam`); decoupled decay $\theta \leftarrow (1 - \eta\lambda)\theta$ (`adamw`)
`schedule`	`"none"`	`"cosine"`: $\eta \to 0$ over the run, per step; `"step"`: $\eta/10$ at 50% and 75% of the run
`warmup_epochs`	`0`	linear warmup of η from 0 over this many epochs
`init`	`"default"`	`"default"` (PyTorch: $\mathrm{Var}\, W = 1/3n_{\text{in}}$), `"he"` ($2/n_{\text{in}}$), `"xavier"` ($1/n_{\text{in}}$)
`batchnorm`	`false`	BatchNorm after every hidden linear layer, before the activation
`dropout`	`0.0`	as in Part B
`augment`	`false`	every training image: random crop (after padding by 4 pixels) and a horizontal flip with probability 1/2, fresh every step

`history` has one new entry, `lr`: the learning rate at the end of each epoch. Everything else (`hidden`, `activation`, `batch_size`, `epochs`, `seed`, `test`, ...) is as before.

Copy `mlp_lab2.py` into the same folder (same Python setup, same `./data`). Check:
`python -c "import mlp_lab2; mlp_lab2.train({'epochs': 1, 'optimizer': 'adam', 'lr': 1e-3})"`
Adam costs about 1.5× the time per epoch of plain SGD, augmentation about 2.5×.

The experiments

Four experiments, about 8 minutes of compute in total. Predict, run, look, explain.

E12. Optimizers

~2.5 min

Run. 15 epochs each, default network: plain SGD at $\eta = 0.05$ (the baseline); `momentum` $= 0.9$ with $\eta = 0.005$; `optimizer` $=$ `"adam"` with $\eta = 10^{-4}$ and with $\eta = 10^{-3}$.

Collect. training loss vs. epoch and validation accuracy vs. epoch, four runs on one axis each; a table with `val_acc_at_best`, `max_val_acc`, `best_epoch`, `final_train_loss`.

Answer. With momentum β and a constant gradient the step is $\eta/(1-\beta)$: which run should match plain SGD at 0.05, and does it? Which run drives the training loss down fastest, and what does its validation loss do meanwhile? Rank the four by `val_acc_at_best`: is any difference larger than E1's spread?

E13. Depth, fixed

~2 min

Run. `hidden` $=$ `[256]*8`, 15 epochs, `activation` $\in$ {`"sigmoid"`, `"relu"`} $\times$ two fixes: `init` $=$ `"he"`, and `batchnorm` $=$ `true`. E7 (no fix) is the baseline.

Collect. training loss vs. epoch, the four runs and E7's two on one axis; `max_val_acc` in a table.

Answer. Which runs train now, and from which epoch? He initialization keeps the signal and the gradient of size $O(1)$ through ReLU layers; why does it not rescue the sigmoid network? What does BatchNorm do to the sigmoid network that an initialization cannot?

E14. Data augmentation

~2 min

Run. default network, 30 epochs, `augment` $=$ `true`. Compare with the first 30 epochs of E4 (same config without augmentation; rerun it at 30 epochs if you do not have E4).

Collect. training and validation accuracy vs. epoch, and validation loss vs. epoch, both runs on the same axes.

Answer. Does the validation loss still turn upward? The training accuracy no longer climbs to 100%: the network is the same, so what changed? How large is the gain in `val_acc_at_best` compared with E1's spread, and compared with weight decay (E9) and dropout (E10)? Why is augmentation a statement about the *data* rather than about the network?

E15. Everything together, test once

~3 min

Run. 30 epochs with `schedule` $=$ `"cosine"` and `augment` $=$ `true`; two configs: (a) AdamW, $\eta = 10^{-3}$, `weight_decay` $= 0.05$; (b) SGD with `momentum` 0.9, $\eta = 0.005$.

Collect. validation accuracy vs. epoch for both, with E14 on the same axis; the `lr` history of one run; a table: config, `val_acc_at_best`, `max_val_acc`, `best_epoch`, `final_train_acc`.

Choose the better config by validation accuracy only. Run it **once** more with `test = true` and report the test accuracy.

Answer. Which epoch is the best one now, and why is that different from E4? How far above E11's test number are you, and what brought most of the gain: the optimizer (E12), the schedule, or the augmentation (E14)? The network is still an MLP on raw pixels. What limits it now? (The next lab.)